

Algorithms and Satisfiability

4. Mini-Project: Satisfiability Modulo Theories

Solving Problems using Satisfiability

Álvaro Torralba



AALBORG UNIVERSITET

Spring 2025

Agenda

- 1 Introduction
- 2 Encoding Problems as SAT
- 3 Predicate Logic
- 4 SAT Modulo Theories
- 5 Practical Considerations
- 6 Beyond

Agenda

- 1 Introduction
- 2 Encoding Problems as SAT
- 3 Predicate Logic
- 4 SAT Modulo Theories
- 5 Practical Considerations
- 6 Beyond

Reminder: The SAT Problem

SAT

Is a **propositional logic** formula ϕ **satisfiable**?

- **Propositional Logic**: Satisfiability can refer to many different logics. In this course, we focus on one of the simplest!
- **Satisfiable**: A formula is satisfiable if it is possible to find an interpretation (assignment) that makes the formula true.

→ Does there exist an assignment that makes the formula true?

Examples:

- $(x \vee y) \wedge (\neg x \vee \neg y)$, Yes! $x = 0, y = 1$
- $(x \vee y) \wedge (\neg x) \wedge (\neg y)$, No!

Reminder: General Problem Solving using Satisfiability

(some new problem)



model problem in logic $\mapsto$ use off-the-shelf reasoning tool



(its solution)

- “Any problem that can be formulated as reasoning about logic.”
- Very successful using propositional logic and modern solvers for SAT! (Propositional satisfiability testing)

→ **This mini-project is about how to encode problems as SAT in practice**

Our Agenda for This Chapter

How to use satisfiability tools to solve practical problems?

- 1 Encode problem as a logical formula
- 2 Use SAT solver

Our Agenda for This Chapter

How to use satisfiability tools to solve practical problems?

- 1 Encode problem as a logical formula
- 2 Use SAT solver

Apart from propositional logic, modern solvers also support more expressive formalisms:

- **Predicate Logic:** Beyond propositional logic
→Introducing quantifiers in our formulas!

Our Agenda for This Chapter

How to use satisfiability tools to solve practical problems?

- 1 Encode problem as a logical formula
- 2 Use SAT solver

Apart from propositional logic, modern solvers also support more expressive formalisms:

- **Predicate Logic:** Beyond propositional logic
→Introducing quantifiers in our formulas!
- **SAT Modulo Theories:** Extending Boolean logic with an external theory
→Increases the expressiveness of the language supported by the solver!

Our Agenda for This Chapter

How to use satisfiability tools to solve practical problems?

- 1 Encode problem as a logical formula
- 2 Use SAT solver

Apart from propositional logic, modern solvers also support more expressive formalisms:

- **Predicate Logic:** Beyond propositional logic
→Introducing quantifiers in our formulas!
- **SAT Modulo Theories:** Extending Boolean logic with an external theory
→Increases the expressiveness of the language supported by the solver!
- **Practical Considerations:** How to encode your problem!
→Some quick tips!

Agenda

- 1 Introduction
- 2 Encoding Problems as SAT
- 3 Predicate Logic
- 4 SAT Modulo Theories
- 5 Practical Considerations
- 6 Beyond

Encoding Problems as SAT

- ❶ Step 1: What are your decision variables?
 - You should be able to read the solution from them!
 - You may also need “auxiliary variables”, that allow you to express your constraints
- ❷ Step 2: Produce the formula
 - All elements in your problem should be reflected in the formula in one way or another
- ❸ Step 3: How exactly satisfiability solves our problem?
 - Sometimes, our problem is about finding a satisfying assignment
 - Sometimes, our problem is about showing that the formula is unsatisfiable

Reasoning in the Wumpus World

A: Agent, **V:** Visited, **OK:** Safe, **P:** Pit, **W:** Wumpus, **B:** Breeze, **S:** Stench, **G:** Gold

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2 OK	2,2	3,2	4,2
1,1 A OK	2,1 OK	3,1	4,1

(1) Initial state

Reasoning in the Wumpus World

A: Agent, **V:** Visited, **OK:** Safe, **P:** Pit, **W:** Wumpus, **B:** Breeze, **S:** Stench, **G:** Gold

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2 OK	2,2	3,2	4,2
1,1 A OK	2,1 OK	3,1	4,1

(1) Initial state

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2 OK	2,2 P?	3,2	4,2
1,1 V OK	2,1 A B OK	3,1 P?	4,1

(2) One step to right

Reasoning in the Wumpus World

A: Agent, **V:** Visited, **OK:** Safe, **P:** Pit, **W:** Wumpus, **B:** Breeze, **S:** Stench, **G:** Gold

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2	2,2	3,2	4,2
1,1 A OK	2,1 OK	3,1	4,1

(1) Initial state

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2	2,2 P?	3,2	4,2
1,1 V OK	2,1 A B OK	3,1 P?	4,1

(2) One step to right

1,4	2,4	3,4	4,4
1,3 W!	2,3	3,3	4,3
1,2 A S OK	2,2 OK	3,2	4,2
1,1 V OK	2,1 B V OK	3,1 P!	4,1

(3) Back, and up to [1,2]

→ The Wumpus is in [1,3]! How do we know?

Reasoning in the Wumpus World

A: Agent, **V:** Visited, **OK:** Safe, **P:** Pit, **W:** Wumpus, **B:** Breeze, **S:** Stench, **G:** Gold

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2	2,2	3,2	4,2
1,1 OK A OK	2,1 OK	3,1	4,1

(1) Initial state

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2	2,2 P?	3,2	4,2
1,1 V OK	2,1 A B OK	3,1 P?	4,1

(2) One step to right

1,4	2,4	3,4	4,4
1,3 W!	2,3	3,3	4,3
1,2 A S OK	2,2 OK	3,2	4,2
1,1 V OK	2,1 B V OK	3,1 P!	4,1

(3) Back, and up to [1,2]

→ The Wumpus is in [1,3]! How do we know? Because in [2,1] we perceived no Stench, the Stench in [1,2] can only come from [1,3].

Reasoning in the Wumpus World

A: Agent, **V:** Visited, **OK:** Safe, **P:** Pit, **W:** Wumpus, **B:** Breeze, **S:** Stench, **G:** Gold

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2	2,2	3,2	4,2
1,1 A OK	2,1 OK	3,1	4,1

(1) Initial state

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2	2,2 P?	3,2	4,2
1,1 V OK	2,1 A B OK	3,1 P?	4,1

(2) One step to right

1,4	2,4	3,4	4,4
1,3 W!	2,3	3,3	4,3
1,2 A S OK	2,2 OK	3,2	4,2
1,1 V OK	2,1 B V OK	3,1 P!	4,1

(3) Back, and up to [1,2]

→ The Wumpus is in [1,3]! How do we know? Because in [2,1] we perceived no Stench, the Stench in [1,2] can only come from [1,3].

→ There's a Pit in [3,1]! How do we know?

Reasoning in the Wumpus World

A: Agent, **V:** Visited, **OK:** Safe, **P:** Pit, **W:** Wumpus, **B:** Breeze, **S:** Stench, **G:** Gold

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2	2,2	3,2	4,2
1,1 A OK	2,1 OK	3,1	4,1

(1) Initial state

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2	2,2 P?	3,2	4,2
1,1 V OK	2,1 A B OK	3,1 P?	4,1

(2) One step to right

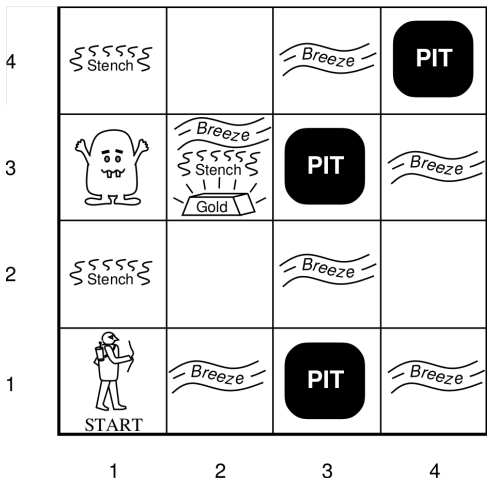
1,4	2,4	3,4	4,4
1,3 W!	2,3	3,3	4,3
1,2 A S OK	2,2 OK	3,2	4,2
1,1 V OK	2,1 B V OK	3,1 P!	4,1

(3) Back, and up to [1,2]

→ **The Wumpus is in [1,3]! How do we know?** Because in [2,1] we perceived no Stench, the Stench in [1,2] can only come from [1,3].

→ **There's a Pit in [3,1]! How do we know?** Because in [1,2] we perceived no Breeze, the Breeze in [2,1] can only come from [3,1].

Where is the Wumpus? The Situation



Where is the Wumpus? The Situation

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2 A S OK	2,2	3,2	4,2
1,1 V OK	2,1 B V OK	3,1	4,1

Where is the Wumpus? Our Knowledge

→ We worry only about the Wumpus and Stench ...

$S_{i,j}$ = Stench in (i,j) , $W_{i,j}$ = Wumpus in (i,j) .

Propositions whose value we know:

$\neg S_{1,1}$, $\neg W_{1,1}$, $\neg S_{2,1}$, $\neg W_{2,1}$, $S_{1,2}$, $\neg W_{1,2}$

Where is the Wumpus? Our Knowledge

→ We worry only about the Wumpus and Stench ...

$S_{i,j}$ = Stench in (i, j) , $W_{i,j}$ = Wumpus in (i, j) .

Propositions whose value we know:

$\neg S_{1,1}$, $\neg W_{1,1}$, $\neg S_{2,1}$, $\neg W_{2,1}$, $S_{1,2}$, $\neg W_{1,2}$

Knowledge about the wumpus and smell: From “Cell adjacent to Wumpus: Stench (else: None)”, we get, amongst many others:

$S_{1,2} \leftrightarrow (W_{1,3} \vee W_{2,2} \vee W_{1,1})$

...

Where is the Wumpus? Our Knowledge

→ We worry only about the Wumpus and Stench ...

$S_{i,j}$ = Stench in (i,j) , $W_{i,j}$ = Wumpus in (i,j) .

Propositions whose value we know:

$\neg S_{1,1}$, $\neg W_{1,1}$, $\neg S_{2,1}$, $\neg W_{2,1}$, $S_{1,2}$, $\neg W_{1,2}$

Knowledge about the wumpus and smell: From “Cell adjacent to Wumpus: Stench (else: None)”, we get, amongst many others:

$S_{1,2} \leftrightarrow (W_{1,3} \vee W_{2,2} \vee W_{1,1})$

...

If $\phi \wedge \neg W_{1,3}$ is unsatisfiable then the Wumpus must be at position 1,3!

Issues with Propositional Logic

Awkward to write for humans: E.g., to model the Wumpus world we had to make a copy of the rules for every cell . . .

$$R_1 : \neg S_{1,1} \rightarrow \neg W_{1,1} \wedge \neg W_{1,2} \wedge \neg W_{2,1}$$

$$R_2 : \neg S_{2,1} \rightarrow \neg W_{1,1} \wedge \neg W_{2,1} \wedge \neg W_{2,2} \wedge \neg W_{3,1}$$

$$R_3 : \neg S_{1,2} \rightarrow \neg W_{1,1} \wedge \neg W_{1,2} \wedge \neg W_{2,2} \wedge \neg W_{1,3}$$

Issues with Propositional Logic

Awkward to write for humans: E.g., to model the Wumpus world we had to make a copy of the rules for every cell ...

$$R_1 : \neg S_{1,1} \rightarrow \neg W_{1,1} \wedge \neg W_{1,2} \wedge \neg W_{2,1}$$

$$R_2 : \neg S_{2,1} \rightarrow \neg W_{1,1} \wedge \neg W_{2,1} \wedge \neg W_{2,2} \wedge \neg W_{3,1}$$

$$R_3 : \neg S_{1,2} \rightarrow \neg W_{1,1} \wedge \neg W_{1,2} \wedge \neg W_{2,2} \wedge \neg W_{1,3}$$

Compared to “Cell adjacent to Wumpus: Stench (else: None)”, that is not a very nice description language ...

Issues with Propositional Logic

Awkward to write for humans: E.g., to model the Wumpus world we had to make a copy of the rules for every cell ...

$$R_1 : \neg S_{1,1} \rightarrow \neg W_{1,1} \wedge \neg W_{1,2} \wedge \neg W_{2,1}$$

$$R_2 : \neg S_{2,1} \rightarrow \neg W_{1,1} \wedge \neg W_{2,1} \wedge \neg W_{2,2} \wedge \neg W_{3,1}$$

$$R_3 : \neg S_{1,2} \rightarrow \neg W_{1,1} \wedge \neg W_{1,2} \wedge \neg W_{2,2} \wedge \neg W_{1,3}$$

Compared to “Cell adjacent to Wumpus: Stench (else: None)”, that is not a very nice description language ...

Can we design a more human-like logic?

Issues with Propositional Logic

Awkward to write for humans: E.g., to model the Wumpus world we had to make a copy of the rules for every cell ...

$$R_1 : \neg S_{1,1} \rightarrow \neg W_{1,1} \wedge \neg W_{1,2} \wedge \neg W_{2,1}$$

$$R_2 : \neg S_{2,1} \rightarrow \neg W_{1,1} \wedge \neg W_{2,1} \wedge \neg W_{2,2} \wedge \neg W_{3,1}$$

$$R_3 : \neg S_{1,2} \rightarrow \neg W_{1,1} \wedge \neg W_{1,2} \wedge \neg W_{2,2} \wedge \neg W_{1,3}$$

Compared to “Cell adjacent to Wumpus: Stench (else: None)”, that is not a very nice description language ...

Can we design a more human-like logic? Yep:

- **Predicate logic:** Quantification of variables ranging over objects.
- ... and a whole zoo of logics much more powerful still.

Issues with Propositional Logic

Awkward to write for humans: E.g., to model the Wumpus world we had to make a copy of the rules for every cell ...

$$R_1 : \neg S_{1,1} \rightarrow \neg W_{1,1} \wedge \neg W_{1,2} \wedge \neg W_{2,1}$$

$$R_2 : \neg S_{2,1} \rightarrow \neg W_{1,1} \wedge \neg W_{2,1} \wedge \neg W_{2,2} \wedge \neg W_{3,1}$$

$$R_3 : \neg S_{1,2} \rightarrow \neg W_{1,1} \wedge \neg W_{1,2} \wedge \neg W_{2,2} \wedge \neg W_{1,3}$$

Compared to “Cell adjacent to Wumpus: Stench (else: None)”, that is not a very nice description language ...

Can we design a more human-like logic? Yep:

- **Predicate logic:** Quantification of variables ranging over objects.
- ... and a whole zoo of logics much more powerful still.
- Note: In applications, propositional CNF encodings are generated by computer programs. This mitigates (but does not remove!) the inconveniences of propositional modeling.

Agenda

- 1 Introduction
- 2 Encoding Problems as SAT
- 3 Predicate Logic**
- 4 SAT Modulo Theories
- 5 Practical Considerations
- 6 Beyond

Let's Talk About Blocks, Baby ...

Dear students: What do you see here?



You say:

Let's Talk About Blocks, Baby ...

Dear students: What do you see here?



You say: “All blocks are red”; “All blocks are on the table”; “A is a block”.

Let's Talk About Blocks, Baby ...

Dear students: What do you see here?



You say: “All blocks are red”; “All blocks are on the table”; “A is a block”.

And now: Say it in propositional logic!

→ “isRedA”, “isRedB”, ..., “onTableA”, “onTableB”, ..., “isBlockA”, ...

Let's Talk About Blocks, Baby ...

Dear students: What do you see here?



You say: “All blocks are red”; “All blocks are on the table”; “A is a block”.

And now: Say it in propositional logic!

→ “isRedA”, “isRedB”, ..., “onTableA”, “onTableB”, ..., “isBlockA”, ...

Wait a sec! Why don't we just say, e.g., “AllBlocksAreRed” and “isBlockA”?

→ Could we conclude that A is red?

Let's Talk About Blocks, Baby ...

Dear students: What do you see here?



You say: “All blocks are red”; “All blocks are on the table”; “A is a block”.

And now: Say it in propositional logic!

→ “isRedA”, “isRedB”, ..., “onTableA”, “onTableB”, ..., “isBlockA”, ...

Wait a sec! Why don't we just say, e.g., “AllBlocksAreRed” and “isBlockA”?

→ **Could we conclude that A is red?** No. These statements are atomic (just strings); their inner structure (“all blocks”, “is a block”) is not captured.

Let's Talk About Blocks, Baby ...

Dear students: What do you see here?



You say: “All blocks are red”; “All blocks are on the table”; “A is a block”.

And now: Say it in propositional logic!

→ “isRedA”, “isRedB”, ..., “onTableA”, “onTableB”, ..., “isBlockA”, ...

Wait a sec! Why don't we just say, e.g., “AllBlocksAreRed” and “isBlockA”?

→ **Could we conclude that A is red?** No. These statements are atomic (just strings); their inner structure (“all blocks”, “is a block”) is not captured.

→ Predicate Logic extends propositional logic with the ability to explicitly speak about objects and their properties.

→ Variables ranging over objects, predicates describing object properties, ...

→ “ $\forall x [Block(x) \rightarrow Red(x)]$ ”; “ $Block(A)$ ”

→ We consider **first-order logic**, and will abbreviate **PL1**.

The Alphabet of PL1

Common symbols:

- **Variables:** $x, x_1, x_2, \dots, x', x'', \dots, y, \dots, z, \dots$
- **Truth/Falseness:** $\top, \perp$. (As in propositional logic)
- **Operators:** $\neg, \vee, \wedge, \rightarrow, \leftrightarrow$. (As in propositional logic)
- **Quantifiers:** $\forall, \exists$.
→ Precedence: $\neg > \forall, \exists > \dots$ (we'll be using brackets).

The Alphabet of PL1

Common symbols:

- **Variables:** $x, x_1, x_2, \dots, x', x'', \dots, y, \dots, z, \dots$
- **Truth/Falseness:** $\top, \perp$. (As in propositional logic)
- **Operators:** $\neg, \vee, \wedge, \rightarrow, \leftrightarrow$. (As in propositional logic)
- **Quantifiers:** $\forall, \exists$.
→ Precedence: $\neg > \forall, \exists > \dots$ (we'll be using brackets).

Application-specific symbols:

- **Constant symbols** (“object”, e.g., $BlockA, BlockB, a, b, c, \dots$)
- **Predicate symbols, arity ≥ 1** (e.g., $Block(.), Above(.,.)$)
- **Function symbols, arity ≥ 1** (e.g., $WeightOf(.), Succ(.)$)

Definition (Signature). A *signature* Σ in predicate logic is a finite set of constant symbols, predicate symbols, and function symbols.

Our “Silly Running Example”: Lassie & Garfield

Constant symbols: *Lassie, Garfield, Snoopy, Lasagna, ...*

Predicate symbols: *Dog(.)*, *Cat(.)*, *Eats(.,.)*, *Chases(.,.)*, ...

Function symbols: *FoodOf(.)*, ...

Example: $\forall x [Dog(x) \rightarrow \exists y Chases(x, y)]$, which in words means

Our “Silly Running Example”: Lassie & Garfield

Constant symbols: *Lassie, Garfield, Snoopy, Lasagna, ...*

Predicate symbols: *Dog(.), Cat(.), Eats(.,.), Chases(.,.), ...*

Function symbols: *FoodOf(.), ...*

Example: $\forall x [Dog(x) \rightarrow \exists y Chases(x, y)]$, which in words means “Every dog chases something”.

[We'll be showing the Lassie & Garfield example in this color and square brackets all over the place.]

Syntax of PL1

→ Terms represent objects:

Definition (Term). Let Σ be a signature. Then:

1. Every variable and every constant symbol is a Σ -term. $[x, \text{Garfield}]$
2. If $t_1, t_2, \dots, t_n$ are Σ -terms and $f \in \Sigma$ is an n -ary function symbol, then $f(t_1, t_2, \dots, t_n)$ also is a Σ -term. $[\text{FoodOf}(x)]$

Terms without variables are *ground terms*. $[\text{FoodOf}(\text{Garfield})]$

→ For simplicity, we usually don't write the " Σ ".

Syntax of PL1

→ Terms represent objects:

Definition (Term). Let Σ be a signature. Then:

1. Every variable and every constant symbol is a Σ -term. $[x, \text{Garfield}]$
2. If $t_1, t_2, \dots, t_n$ are Σ -terms and $f \in \Sigma$ is an n -ary function symbol, then $f(t_1, t_2, \dots, t_n)$ also is a Σ -term. $[\text{FoodOf}(x)]$

Terms without variables are *ground terms*. $[\text{FoodOf}(\text{Garfield})]$

→ For simplicity, we usually don't write the " Σ -".

→ Atoms represent atomic statements about objects:

Definition (Atom). Let Σ be a signature. Then:

1. $\top$ and $\perp$ are Σ -atoms.
2. If $t_1, t_2, \dots, t_n$ are terms and $P \in \Sigma$ is an n -ary predicate symbol, then $P(t_1, t_2, \dots, t_n)$ is a Σ -atom. $[\text{Chases}(\text{Lassie}, y)]$

Atoms without variables are *ground atoms*. $[\text{Chases}(\text{Lassie}, \text{Garfield})]$

Syntax of PL1, ctd.

→ Formulas represent complex statements about objects:

Definition (Formula). Let Σ be a signature. Then:

1. Each Σ -atom is a Σ -formula.
2. If φ is a Σ -formula, then so is $\neg\varphi$.

The formulas that can be constructed by rules 1. and 2. are *literals*.

Syntax of PL1, ctd.

→ Formulas represent complex statements about objects:

Definition (Formula). Let Σ be a signature. Then:

1. Each Σ -atom is a Σ -formula.
2. If φ is a Σ -formula, then so is $\neg\varphi$.

The formulas that can be constructed by rules 1. and 2. are *literals*.

If φ and ψ are Σ -formulas, then so are:

4. $\varphi \wedge \psi$, $\varphi \vee \psi$, $\varphi \rightarrow \psi$, and $\varphi \leftrightarrow \psi$.

Syntax of PL1, ctd.

→ Formulas represent complex statements about objects:

Definition (Formula). Let Σ be a signature. Then:

1. Each Σ -atom is a Σ -formula.
2. If φ is a Σ -formula, then so is $\neg\varphi$.

The formulas that can be constructed by rules 1. and 2. are *literals*.

If φ and ψ are Σ -formulas, then so are:

4. $\varphi \wedge \psi$, $\varphi \vee \psi$, $\varphi \rightarrow \psi$, and $\varphi \leftrightarrow \psi$.

If φ is a Σ -formula and x is a variable, then

5. $\forall x\varphi$ is a Σ -formula (“Universal Quantification”).
6. $\exists x\varphi$ is a Σ -formula (“Existential Quantification”).

Syntax of PL1, ctd.

→ Formulas represent complex statements about objects:

Definition (Formula). Let Σ be a signature. Then:

1. Each Σ -atom is a Σ -formula.
2. If φ is a Σ -formula, then so is $\neg\varphi$.

The formulas that can be constructed by rules 1. and 2. are *literals*.

If φ and ψ are Σ -formulas, then so are:

4. $\varphi \wedge \psi$, $\varphi \vee \psi$, $\varphi \rightarrow \psi$, and $\varphi \leftrightarrow \psi$.

If φ is a Σ -formula and x is a variable, then

5. $\forall x\varphi$ is a Σ -formula (“Universal Quantification”).
6. $\exists x\varphi$ is a Σ -formula (“Existential Quantification”).

→ [E.g., $Cat(Garfield) \vee \neg Cat(Garfield)$; and $\exists x[Eats(Garfield, x)]$]

Questionnaire

Example “Animals” Σ : Constant symbols

$\{Lassie, Garfield, Snoopy, Lasagna\}$; predicate symbols $\{Dog(.), Cat(.), Eats(.,.), Chases(.,.)\}$; function symbols $\{FoodOf(.)\}$.

Question!

Which of these are Σ -formulas?

(A):

$Eats(Garfield, FoodOf(Lassie))$

(B): $Eats(Snoopy, Cat(Garfield))$

(C): $\forall x[(Dog(x) \wedge Eats(x, Lasagna)) \rightarrow \exists y(Cat(y) \wedge Chases(y, x))]$

(D): $\exists x[Dog(x) \wedge Eats(x, Lasagna) \wedge \forall y(Cat(y) \rightarrow Chases(y, x))]$

Questionnaire

Example “Animals” Σ : Constant symbols

$\{Lassie, Garfield, Snoopy, Lasagna\}$; predicate symbols $\{Dog(.), Cat(.), Eats(.,.), Chases(.,.)\}$; function symbols $\{FoodOf(.)\}$.

Question!

Which of these are Σ -formulas?

(A):

$Eats(Garfield, FoodOf(Lassie))$ (B): $Eats(Snoopy, Cat(Garfield))$

(C): $\forall x[(Dog(x) \wedge Eats(x, Lasagna)) \rightarrow \exists y(Cat(y) \wedge Chases(y, x))]$

(D): $\exists x[Dog(x) \wedge Eats(x, Lasagna) \forall y(Cat(y) \rightarrow Chases(y, x))]$

→ (A), (C): Yes.

→ (B): No, we can't nest predicates.

→ (D): No, missing a connective between “ $Eats(x, Lasagna)$ ” and “ $\forall y(Cat(y) \rightarrow Chases(y, x))$ ”.

The Meaning of PL1 Formulas

Example: $\forall x[Block(x) \rightarrow Red(x)], Block(A)$

→ For all objects x , if x is a block, then x is red. A is a block.

The Meaning of PL1 Formulas

Example: $\forall x[Block(x) \rightarrow Red(x)], Block(A)$

→ For all objects x , if x is a block, then x is red. A is a block.

More generally: (Intuition)

- Terms represent objects. $[FoodOf(Garfield) = Lasagna]$
- Predicates represent relations on the universe.
 $[Dog = \{Lassie, Snoopy\}]$
- Universally-quantified variables: “for all objects in the universe”.
- Existentially-quantified variables: “at least one object in the universe”.

PL1 as Syntactic Sugar

Whenever:

- There are no functions
- We have a finite universe Σ

We can transform a PL1 formula to an equivalent propositional formula:

- 1 Replace $\forall x P(x)$ by $P(a) \wedge P(b) \wedge \dots$
- 2 Replace $\exists x P(x)$ by $P(a) \vee P(b) \vee \dots$

PL1 as Syntactic Sugar

Whenever:

- There are no functions
- We have a finite universe Σ

We can transform a PL1 formula to an equivalent propositional formula:

- 1 Replace $\forall x P(x)$ by $P(a) \wedge P(b) \wedge \dots$
- 2 Replace $\exists x P(x)$ by $P(a) \vee P(b) \vee \dots$

Example: $\forall x [Block(x) \rightarrow Red(x)] \wedge Block(A) \wedge Block(B) \wedge Block(C) \wedge Block(D) \wedge Block(E)$

PL1 as Syntactic Sugar

Whenever:

- There are no functions
- We have a finite universe Σ

We can transform a PL1 formula to an equivalent propositional formula:

- 1 Replace $\forall x P(x)$ by $P(a) \wedge P(b) \wedge \dots$
- 2 Replace $\exists x P(x)$ by $P(a) \vee P(b) \vee \dots$

Example: $\forall x [Block(x) \rightarrow Red(x)] \wedge Block(A) \wedge Block(B) \wedge Block(C) \wedge Block(D) \wedge Block(E)$

Equivalent:

PL1 as Syntactic Sugar

Whenever:

- There are no functions
- We have a finite universe Σ

We can transform a PL1 formula to an equivalent propositional formula:

- 1 Replace $\forall x P(x)$ by $P(a) \wedge P(b) \wedge \dots$
- 2 Replace $\exists x P(x)$ by $P(a) \vee P(b) \vee \dots$

Example: $\forall x [Block(x) \rightarrow Red(x)] \wedge Block(A) \wedge Block(B) \wedge Block(C) \wedge Block(D) \wedge Block(E)$

Equivalent: $(Block(A) \rightarrow Red(A)) \wedge (Block(B) \rightarrow Red(B)) \wedge (Block(C) \rightarrow Red(C)) \wedge (Block(D) \rightarrow Red(D)) \wedge (Block(E) \rightarrow Red(E)) \wedge Block(A) \wedge Block(B) \wedge Block(C) \wedge Block(D) \wedge Block(E)$

→ But PL1 is much more expressive in general!

Blocks, Who Cares? Let's Talk About Numbers!

Example “Integers”: (A limited vocabulary to talk about these)

- The objects: $\{1, 2, 3, \dots\}$.
- Predicate 1: “ $Even(x)$ ” should be true iff x is even.
- Predicate 2: “ $Equals(x, y)$ ” should be true iff $x = y$.
- Function: $Succ(x)$ maps x to $x + 1$.

Blocks, Who Cares? Let's Talk About Numbers!

Example “Integers”: (A limited vocabulary to talk about these)

- The objects: $\{1, 2, 3, \dots\}$.
- Predicate 1: “ $Even(x)$ ” should be true iff x is even.
- Predicate 2: “ $Equals(x, y)$ ” should be true iff $x = y$.
- Function: $Succ(x)$ maps x to $x + 1$.

Old problem: Say, in propositional logic, that “ $1 + 1 = 2$ ”.

→ Inner structure of vocabulary is ignored (cf. “AllBlocksAreRed”).

→ PL1 solution: “ $Equals(Succ(1), 2)$ ”.

Blocks, Who Cares? Let's Talk About Numbers!

Example “Integers”: (A limited vocabulary to talk about these)

- The objects: $\{1, 2, 3, \dots\}$.
- Predicate 1: “*Even*(x)” should be true iff x is even.
- Predicate 2: “*Equals*(x, y)” should be true iff $x = y$.
- Function: *Succ*(x) maps x to $x + 1$.

Old problem: Say, in propositional logic, that “ $1 + 1 = 2$ ”.

→ Inner structure of vocabulary is ignored (cf. “AllBlocksAreRed”).

→ PL1 solution: “*Equals*(*Succ*(1), 2)”.

New problem: Say, in propositional logic, “if x is even, so is $x + 2$ ”.

Blocks, Who Cares? Let's Talk About Numbers!

Example “Integers”: (A limited vocabulary to talk about these)

- The objects: $\{1, 2, 3, \dots\}$.
- Predicate 1: “ $Even(x)$ ” should be true iff x is even.
- Predicate 2: “ $Equals(x, y)$ ” should be true iff $x = y$.
- Function: $Succ(x)$ maps x to $x + 1$.

Old problem: Say, in propositional logic, that “ $1 + 1 = 2$ ”.

→ Inner structure of vocabulary is ignored (cf. “AllBlocksAreRed”).

→ PL1 solution: “ $Equals(Succ(1), 2)$ ”.

New problem: Say, in propositional logic, “if x is even, so is $x + 2$ ”.

→ It is impossible to speak about infinite sets of objects!

→ PL1 solution: “ $\forall x[Even(x) \rightarrow Even(Succ(Succ(x)))]$ ”.

Now We're Talking ...

$$\begin{aligned} \forall y, x_1, x_2, x_3 \ [& \text{Equals}(\text{Plus}(\text{PowerOf}(x_1, y), \text{PowerOf}(x_2, y)), \\ & \text{PowerOf}(x_3, y)) \\ & \rightarrow (\text{Equals}(y, 1) \vee \text{Equals}(y, 2))] \end{aligned}$$

Now We're Talking ...

$$\begin{aligned} \forall y, x_1, x_2, x_3 \ [& \text{Equals}(\text{Plus}(\text{PowerOf}(x_1, y), \text{PowerOf}(x_2, y)), \\ & \text{PowerOf}(x_3, y)) \\ & \rightarrow (\text{Equals}(y, 1) \vee \text{Equals}(y, 2))] \end{aligned}$$

Theorem proving in PL1! Arbitrary theorems, in principle.

Fermat's last theorem is of course infeasible, but interesting theorems can and have been proved automatically.

See http://en.wikipedia.org/wiki/Automated_theorem_proving.

Now We're Talking ...

$$\begin{aligned} \forall y, x_1, x_2, x_3 \ [& \text{Equals}(\text{Plus}(\text{PowerOf}(x_1, y), \text{PowerOf}(x_2, y)), \\ & \text{PowerOf}(x_3, y)) \\ & \rightarrow (\text{Equals}(y, 1) \vee \text{Equals}(y, 2))] \end{aligned}$$

Theorem proving in PL1! Arbitrary theorems, in principle.

Fermat's last theorem is of course infeasible, but interesting theorems can and have been proved automatically.

See http://en.wikipedia.org/wiki/Automated_theorem_proving.

Note: Need to **axiomatize** “Plus”, “PowerOf”, “Equals”.

See http://en.wikipedia.org/wiki/Peano_axioms

Agenda

- 1 Introduction
- 2 Encoding Problems as SAT
- 3 Predicate Logic
- 4 SAT Modulo Theories**
- 5 Practical Considerations
- 6 Beyond

SAT Modulo Theories

- **Satisfiability Modulo Theories** (SMT): extend propositional reasoning with one or more **external theories**

SAT Modulo Theories

- **Satisfiability Modulo Theories** (SMT): extend propositional reasoning with one or more **external theories**
- Propositional SAT: Is $(P \vee Q) \wedge (\neg P \vee \neg Q)$ satisfiable?
- Theories:
 - Equality and uninterpreted functions

$$f(x) = f(y) \wedge x \neq y$$

- Arithmetic: rational, integer, real, ...

$$(x < y) \wedge ((y < z) \vee (x < z)) \wedge (z - 5 < x + 2)$$

- Arrays, binary vectors, ...

Solving SMT: CDCL(T) by Example

$$(x < y) \wedge ((y < z) \vee (x < z)) \wedge (z - 5 < x + 2)$$

Solving SMT: CDCL(T) by Example

$$\underbrace{(x < y)}_P \wedge \underbrace{((y < z) \vee (x < z))}_Q \wedge \underbrace{(z - 5 < x + 2)}_S$$

- ① Use Boolean abstraction:

$$P \wedge (Q \vee R) \wedge S$$

Solving SMT: CDCL(T) by Example

$$\underbrace{(x < y)}_P \wedge \underbrace{((y < z) \vee (x < z))}_Q \wedge \underbrace{(z - 5 < x + 2)}_S$$

- ① Use Boolean abstraction:

$$P \wedge (Q \vee R) \wedge S$$

- ② Use CDCL to find a (partial) satisfying assignment:

$$P \mapsto 1, Q \mapsto 1, S \mapsto 1$$

Solving SMT: CDCL(T) by Example

$$\underbrace{(x < y)}_P \wedge \underbrace{((y < z) \vee (x < z))}_Q \wedge \underbrace{(z - 5 < x + 2)}_S$$

- ① Use Boolean abstraction:

$$P \wedge (Q \vee R) \wedge S$$

- ② Use CDCL to find a (partial) satisfying assignment:

$$P \mapsto 1, Q \mapsto 1, S \mapsto 1$$

- ③ Call theory solver to check if it satisfies the external theory: If yes, we are done. Otherwise, learn a new clause: $(\neg P \vee \neg Q \vee \neg S)$

Remark: This clause learning should learn a clause as informative as possible by using a justification procedure as in [Chapter 8](#).

Solving SMT: CDCL(T) by Example

$$\underbrace{(x < y)}_P \wedge \underbrace{((y < z) \vee (x < z))}_Q \wedge \underbrace{(z - 5 < x + 2)}_S$$

- ① Use Boolean abstraction:

$$P \wedge (Q \vee R) \wedge S$$

- ② Use CDCL to find a (partial) satisfying assignment:

$$P \mapsto 1, Q \mapsto 1, S \mapsto 1$$

- ③ Call theory solver to check if it satisfies the external theory: If yes, we are done. Otherwise, learn a new clause: $(\neg P \vee \neg Q \vee \neg S)$

Remark: This clause learning should learn a clause as informative as possible by using a justification procedure as in [Chapter 8](#).

- ④ Go to 2

Useful Resources

SMT Competition: <https://smt-comp.github.io>

SMT-Lib: <https://smtlib.cs.uiowa.edu/>

There are many SMT solvers available. To name a few:

- Z3: <https://github.com/Z3Prover/z3>
- CVC4: <https://cvc4.github.io/>
- Yices 2: <https://github.com/SRI-CSL/yices2>

Format SMT-LIB2

- Choose the theory to be used:

```
(set_logic QF_LIA)
```

- Some theories:
 - QF_LRA: linear real arithmetic without quantifiers
 - QF_LIA: linear integer arithmetic without quantifiers
- SMT-LIB 2 does not allow us to mix problems, although some of the solvers allow it

Format SMT-LIB2

Declare variables:

```
(declare-fun x () Int)
(declare-fun z 1 3 () Real)
```

Add formulas: expressions are written in prefix form:

$(\langle operator \rangle \langle arg1 \rangle \cdots \langle argn \rangle)$

```
(assert (or (<= (+ x 3) (* 2 u))
            (>= (+ v 4) y)
            (>= (+ x y z ) 2)))
```

Format SMT-LIB2

- Ask the solver to verify satisfiability

```
(check-sat)
```

- And report the model

```
(get-model)
```

Agenda

- 1 Introduction
- 2 Encoding Problems as SAT
- 3 Predicate Logic
- 4 SAT Modulo Theories
- 5 Practical Considerations**
- 6 Beyond

How to Prove Something

Typical Exercise: Given X , Y , and Z , prove φ to be true

- Knowledge Base KB: a formula KB that we know to be true
 $(X \wedge Y \wedge Z)$
- Statement that we want to prove: φ

How to Prove Something

Typical Exercise: Given X , Y , and Z , prove φ to be true

- Knowledge Base KB: a formula KB that we know to be true
 $(X \wedge Y \wedge Z)$
- Statement that we want to prove: φ

Proof by contradiction: Assume $\neg\varphi$ and show that this leads to a contradiction

How to Prove Something

Typical Exercise: Given X , Y , and Z , prove φ to be true

- Knowledge Base KB: a formula KB that we know to be true
($X \wedge Y \wedge Z$)
- Statement that we want to prove: φ

Proof by contradiction: Assume $\neg\varphi$ and show that this leads to a contradiction

→ Deduction can be reduced to proving unsatisfiability of $\text{KB} \wedge \neg\varphi$:
“Assume, to the contrary, that KB holds but φ does not hold; then derive False”.

How to Prove Something

Typical Exercise: Given X , Y , and Z , prove φ to be true

- Knowledge Base KB: a formula KB that we know to be true
($X \wedge Y \wedge Z$)
- Statement that we want to prove: φ

Proof by contradiction: Assume $\neg\varphi$ and show that this leads to a contradiction

→ Deduction can be reduced to proving unsatisfiability of $\text{KB} \wedge \neg\varphi$:
“Assume, to the contrary, that KB holds but φ does not hold; then derive False”.

→ You also need to check that KB is satisfiable, otherwise, if there is a contradiction in KB you can prove any statement!

Sometimes Less is More

There are a whole lot of theories, which one should I use?

Sometimes Less is More

There are a whole lot of theories, which one should I use?

Trade-off between Expressiveness and Complexity:

- SAT is NP
- PL1 is undecidable
- SMT depends on the theory

Sometimes Less is More

There are a whole lot of theories, which one should I use?

Trade-off between Expressiveness and Complexity:

- SAT is NP
- PL1 is undecidable
- SMT depends on the theory

→ Always try to express your problem in a way that's as simple as possible! (e.g. propositional SAT without any theories)

The solvers available to reason about arithmetical constraints are wildly different depending on what fragments of arithmetic is used. We summarize the main fragments, available decision procedures, and examples in Table 1 where x, y range over reals and a, b range over integers.

Logic	Description	Solver	Example
LRA	Linear Real Arithmetic	Dual Simplex [28]	$x + \frac{1}{2}y \leq 3$
LIA	Linear Integer Arithmetic	Cuts + Branch	$a + 3b \leq 3$
LIRA	Mixed Real/Integer	[7, 12, 14, 26, 28]	$x + a \geq 4$
IDL	Integer Difference Logic	Floyd-Warshall	$a - b \leq 4$
RDL	Real Difference Logic	Bellman-Ford	$x - y \leq 4$
UTVPI	Unit two-variable / inequality	Bellman-Ford	$x + y \leq 4$
NRA	Polynomial Real Arithmetic	Model based CAD [42]	$x^2 + y^2 < 1$
NIA	Non-linear Integer Arithmetic	CAD + Branch [41]	$a^2 = 2$
	Algorithms and Satisfiability	Linearization [15]	Chapter 4: SMT

Solving Numerical Problems

Linear Programming

- Allows you to handle numerical problems of the form:

$$\begin{array}{ll}\text{minimize} & x + 5y \\ \text{subject to} & x \leq 3 \\ & 2x + y \leq 4\end{array}$$

Solving Numerical Problems

Linear Programming

- Allows you to handle numerical problems of the form:

$$\begin{array}{ll}\text{minimize} & x + 5y \\ \text{subject to} & x \leq 3 \\ & 2x + y \leq 4\end{array}$$

Mixed Integer Linear Programming:

- Allows to define binary (Boolean) and Integer variables
- Can also encode SAT

$$p \vee q \text{ encoded as } x_p + x_q \geq 1$$

Solving Numerical Problems

Linear Programming

- Allows you to handle numerical problems of the form:

$$\begin{array}{ll}\text{minimize} & x + 5y \\ \text{subject to} & x \leq 3 \\ & 2x + y \leq 4\end{array}$$

Mixed Integer Linear Programming:

- Allows to define binary (Boolean) and Integer variables
- Can also encode SAT

$$p \vee q \text{ encoded as } x_p + x_q \geq 1$$

- If your problem is mostly numerical, then LP or MILP are good alternatives too!

Answer Set Programming

Agenda

- 1 Introduction
- 2 Encoding Problems as SAT
- 3 Predicate Logic
- 4 SAT Modulo Theories
- 5 Practical Considerations
- 6 Beyond

Types of Logic

- Propositional logic
- Predicate logic
- Modal Logics
 - Temporal Logics
 - Epistemic Logic
 - Description Logic
- Higher-order logics

Temporal Logics: LTL, CTL, etc.

Temporal logics are used to reason about propositions in terms of time.

Some Temporal Operators:

- **Next (X):** Xp — "p will be true in the next moment."
- **Eventually ($\Diamond$):** $\Diamond p$ — "p will be true at some point in the future."
- **Always ($\Box$):** $\Box p$ — "p will be true at all times in the future."
- **Until (U):** $p U q$ — "p will hold until q becomes true."

Examples:

- Let x_o be "The system is online", x_b means someone pressed the button.
- x_o — "The system is online right now"
- Xx_o — "The system will be online in the next moment."
- $\Diamond x_o$ — "The system will eventually be online."
- $\Box x_o$ — "The system will always be online."
- $x_o U x_b$ — "The system will remain online until someone presses the button."

Epistemic Logic

Epistemic logics are used to reason about knowledge and beliefs of agents.

Key Epistemic Operators:

- **Knows (K):** K_ap — "Agent a knows that p ."
- **Believes (B):** B_ap — "Agent a believes that p ."

Knowledge is typically considered to be true — "if an agent knows p , then p is true, but if an agent believes p , p may not be true."

Examples:

- K_ax_o — Agent a knows that the system is online.

Epistemic Logic

Epistemic logics are used to reason about knowledge and beliefs of agents.

Key Epistemic Operators:

- **Knows (K):** K_ap — "Agent a knows that p ."
- **Believes (B):** B_ap — "Agent a believes that p ."

Knowledge is typically considered to be true — "if an agent knows p , then p is true, but if an agent believes p , p may not be true."

Examples:

- K_ax_o — Agent a knows that the system is online.
- $K_bK_ax_o$ — Agent b knows that agent a knows that the system is online.

Epistemic Logic

Epistemic logics are used to reason about knowledge and beliefs of agents.

Key Epistemic Operators:

- **Knows (K):** K_ap — "Agent a knows that p ."
- **Believes (B):** B_ap — "Agent a believes that p ."

Knowledge is typically considered to be true — "if an agent knows p , then p is true, but if an agent believes p , p may not be true."

Examples:

- K_ax_o — Agent a knows that the system is online.
- $K_bK_ax_o$ — Agent b knows that agent a knows that the system is online.
- $K_aK_bK_ax_o$ — Agent a knows that agent b knows that agent a knows that the system is online.

Epistemic Logic

Epistemic logics are used to reason about knowledge and beliefs of agents.

Key Epistemic Operators:

- **Knows (K):** K_ap — "Agent a knows that p ."
- **Believes (B):** B_ap — "Agent a believes that p ."
- **Common Knowledge (C_A)** C_Ap — p is true, everyone knows p is true, everyone knows that everyone knows that p is true, ...

Knowledge is typically considered to be true — "if an agent knows p , then p is true, but if an agent believes p , p may not be true."

Examples:

- K_ax_o — Agent a knows that the system is online.
- $K_bK_ax_o$ — Agent b knows that agent a knows that the system is online.
- $K_aK_bK_ax_o$ — Agent a knows that agent b knows that agent a knows that the system is online.

Higher-Order Logic

Higher-order logic extends first-order logic by allowing quantification over predicates, functions, and relations, in addition to individuals.

Higher-Order Logic:

- **Quantification over predicates:** e.g., $\forall P(P(x) \rightarrow Q(x))$, where P is a predicate.
- **Quantification over functions:** e.g., $\exists f \forall x(f(x) = y)$, where f is a function.

→ For example, used in Automated Theorem Proving

<https://isabelle.in.tum.de/>